

LAMP: Latent Motion Prior-Guided Real-World Learning for Dexterous Hand Manipulation

Xinye Yang^{1,4,5,*} Zhiyuan Ma^{2,4,*} Hongze Yu^{4,†} Yuanpei Chen⁴
Yaodong Yang³ Xiaojie Chai⁴ Xinlei Chen² Chao Yu^{2,†}

¹Fudan University, ²Tsinghua University, ³Peking University

⁴PsiBot, ⁵Zhongguancun Academy

*Equal contribution, †Corresponding authors

 Project Page

 Code

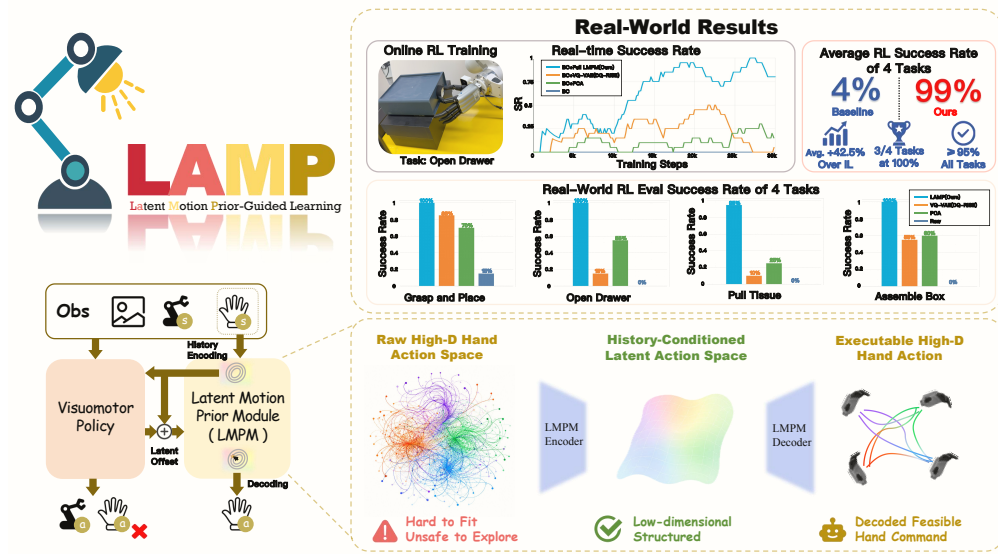


Figure 1: LAMP uses a learned motion prior to expose high-dimensional dexterous hand motion through a compact latent action space, where imitation learning predicts latent offsets and residual reinforcement learning adds latent residuals before decoding executable hand actions.

Abstract: Real-world learning for dexterous hands remains brittle because high-dimensional hand actions amplify imitation errors and make reinforcement-learning exploration prone to contact-breaking motion. While combining imitation learning (IL) with online reinforcement learning (RL) can reduce manual supervision, unconstrained exploration in raw hand-action spaces is sample-inefficient and risky for physical hardware. We introduce a latent motion prior module (LMPM) that maps recent hand-action histories to a compact, history-conditioned latent prior and decodes continuous latent commands into executable high-dimensional hand targets. Built on this prior, LAMP is a three-stage real-world dexterous learning framework: it pretrains LMPM from demonstrations, trains a visuomotor policy that predicts native arm commands and latent hand-action offsets, and improves the policy with online residual RL in the same latent hand-action space. This shared, decodable interface lets residual exploration make local corrections near demonstrated, contact-consistent hand motions rather than perturbing every finger joint independently. We evaluate LAMP on four real-robot dexterous manipulation tasks against raw, linear, and discrete hand-action interfaces. Starting from small task-specific demonstration sets, LAMP achieves a 56.25% average IL success rate and raises it to 98.75% after online RL, reaching 100% final success on three tasks and 95% on the remaining task.

Keywords: Dexterous Manipulation, Motion Prior, Real-World Robot Learning

1 Introduction

Dexterous robot hands promise contact-rich manipulation beyond the capabilities of simple grippers. Their high-dimensional actuation lets them conform to diverse object geometries, adjust grasp postures, and maintain multi-finger contact with articulated or compliant objects. To deploy these capabilities on hardware, current research mainly relies on sim-to-real transfer and imitation learning to bypass the challenges of analytical modeling.

However, both paradigms face severe bottlenecks when scaled to dexterous manipulation. For sim-to-real transfer, the chaotic, discontinuous nature of multi-finger contact dynamics, together with micro-friction and soft-material compliance, makes high-fidelity simulation notoriously difficult, leading to a pronounced reality gap that degrades policy performance upon physical deployment [1]. For imitation learning, standard visuomotor policies [2, 3, 4] typically excel at replicating demonstrated trajectories but fundamentally lack the capacity for active exploration and error correction. Because end-to-end behavioral cloning does not model alternative recovery behaviors, minor execution errors in high-DoF action spaces quickly compound, causing the hand to drift into out-of-distribution (OOD) configurations. Without the exploratory optimization inherent in reinforcement learning, pure imitation learning cannot reason about contact recovery or adapt to dynamic perturbations from limited human demonstrations [5].

To overcome these limitations, online reinforcement learning directly in the real world has emerged as a compelling alternative for fine-tuning behaviors. However, standard RL relies on unconstrained, stochastic exploration to establish reliable action-value estimates. In unconstrained high-DoF spaces, unguided exploration inevitably triggers out-of-domain movements that perturb object poses or break delicate contact patterns. Once the object tilts or falls, mid-episode recovery is practically impossible for a physical hand, leading to irreversible state transitions, extremely sparse successful trajectories, and prohibitively low sample efficiency. This erratic and random exploration contrasts sharply with biological learning. Developmental psychology [6, 7] reveals that human infants do not explore via unstructured, random joint movements; instead, their early actions, known as “structured motor babbling”, are inherently guided by intrinsic neuromuscular priors.

A natural remedy to this exploration crisis lies in exploiting the intrinsic low-dimensional structure of dexterous hand movements. Prior studies suggest that hand motion often contains low-dimensional structure: multi-joint grasping postures can be explained by a small number of modes [8, 9]. This observation has motivated compressed and latent action spaces, from linear hand subspaces to learned continuous or discrete latent policies [10, 11, 12, 5]. Yet a dexterous learning pipeline needs more than dimension reduction: the interface must decode to executable hand targets, support locally smooth residual updates around contact, and use recent motion history so exploration stays near contact-consistent hand motions. This motivates a continuous, decodable, history-conditioned latent action space shared by supervised policy learning and online residual exploration.

To instantiate this interface, we introduce the latent motion prior model (LMPM), which maps recent hand-action history to a distribution in a compact latent action space and decodes sampled latent variables back to raw hand targets. We then build LAMP, a three-stage real-world learning framework around the same prior interface: Stage 1 pretrains LMPM from offline hand-motion data; Stage 2 behavior-clones arm commands and predicts latent hand-action offsets around the LMPM prior; and Stage 3 applies residual reinforcement learning in the same latent action space.

Our contributions are:

- We introduce LMPM, a history-conditioned latent motion prior that learns a continuous, decodable manifold of dexterous hand motion from offline trajectories.
- We develop LAMP, a real-world learning framework that unifies imitation learning and residual reinforcement learning through the shared LMPM latent hand-action interface.
- We validate LAMP on four real-world dexterous tasks, achieving 100% final success on three tasks and 95% on the remaining task, demonstrating stronger real-world learning performance than alternative hand-action interfaces.

2 Related Work

Hand action-space representations. Low-dimensional hand synergies have long been observed in human grasping and tool use [8] and have been used to reduce robotic grasp search [9]. Recent work extends this idea to learned cross-hand pose synergies [13], continuous latent action spaces for offline or online policy search [10, 11], reusable skill priors [14, 15], and shared action spaces from unlabeled or cross-embodiment data [16, 17]. LAMP builds on this continuous representation view, but uses a history-conditioned, decodable hand-action interface as the common control space for both behavior-cloning offsets and online residual-RL corrections on real dexterous tasks.

Discrete representations instead tokenize actions for multimodal behavior modeling and sequence prediction. Examples include vector-quantized behavior policies [12], tokenized or disentangled robot action spaces [18], efficient VLA action tokenization [19], and dexterous policies with discrete action chunks or quantized hand commands [20, 5, 21]. These methods are effective for action chunking and large-scale sequence modeling, but codebook switches can turn small latent changes into abrupt hand-command jumps during contact-rich residual RL. LAMP therefore uses a continuous hand interface constrained by the demonstrated motion manifold.

Real-world imitation and reinforcement learning. Modern real-world imitation learning includes action-chunking transformers [2], diffusion-based visuomotor policies [3], 3D-conditioned diffusion policies [4], and large-scale generalist robot policies [22, 23, 24, 25]. These approaches differ in architecture and data scale, but typically predict environment actions or action chunks directly. Real-world RL systems combine demonstrations, online replay, sparse visual rewards, asynchronous actor-learner infrastructure, and human feedback for data-efficient policy improvement [26, 27, 28, 29, 30, 31]. LAMP changes the action space in which imitation and residual exploration occur within this real-world learning pipeline.

3 Method

LAMP uses LMPM to expose high-dimensional hand control through a compact, history-conditioned latent interface shared by imitation learning and real-world residual RL. In this section, we first learn LMPM as an encoder from hand-target histories to a latent prior and a decoder from latent vectors to executable hand commands (Section 3.1). We then use this frozen prior to train an IL policy that predicts arm commands in the native arm space and hand commands as vision-guided latent offsets around the prior center (Section 3.2). Online residual RL also operates in the latent coordinates, adding native arm residuals and latent hand residuals before decoding the final hand target to avoid contact-breaking perturbations in the original hand-command space (Section 3.3). Figure 2 summarizes the three-stage pipeline.

3.1 Stage 1: Learning the Latent Motion Prior

Stage 1 builds the hand-action interface from the hand-motion component of teleoperated trajectories. Let h_{t+1} denote the target hand command at the next control step, and let

$$H_t = (h_{t-K+1}, \dots, h_t) \quad (1)$$

be the recent hand-target history. The encoder maps H_t to a local latent prior, and the decoder maps latent vectors back to executable hand targets:

$$q_\phi(z_t|H_t) = \mathcal{N}(\mu_\phi(H_t), \text{diag}(\sigma_\phi^2(H_t))) , \quad \hat{h}_{t+1} = D_\theta(z_t), \quad z_t \sim q_\phi(\cdot|H_t). \quad (2)$$

We train the encoder and decoder with a reconstruction objective and a KL-regularized bottleneck:

$$\mathcal{L}_{\text{prior}} = \mathbb{E}_{(H_t, h_{t+1})} [\|D_\theta(z_t) - h_{t+1}\|_2^2 + \beta D_{\text{KL}}(q_\phi(z_t|H_t) \parallel \mathcal{N}(0, I))] . \quad (3)$$

The reconstruction term preserves executability in the original hand-command space, while the KL term encourages a compact and smooth latent coordinate system. After this stage, E_ϕ and D_θ are fixed. The latent action space supplies shared hand-motion coordinates, and the history-conditioned

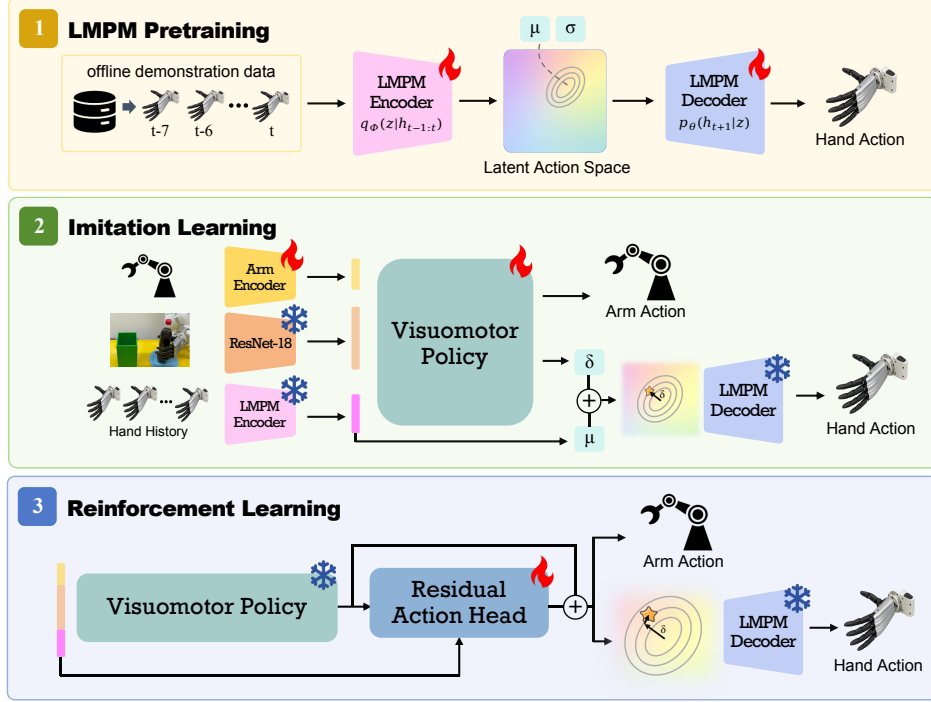


Figure 2: Overview of LAMP. The framework first learns LMPM from demonstrated hand motion and then uses the frozen prior interface in both behavior cloning and real-world residual RL.

distribution supplies the local prior center for the current motion phase. Two histories can share the same latent coordinates yet decode around different hand-motion phases through different encoder statistics.

3.2 Stage 2: Imitation Learning in the Latent Action Space

Stage 2 learns the initial visuomotor policy from demonstrations $\mathcal{D}_{\text{demo}} = \{(o_t, a_t^{\text{arm}}, h_{t+1}, H_t)\}$. At time t , the observation o_t contains RGB images and the arm proprioceptive state x_t^{arm} ; hand motion enters separately through the recent hand-target history H_t used by the frozen prior. The executed action consists of a native arm command and a hand target,

$$a_t = (a_t^{\text{arm}}, a_t^{\text{hand}}), \quad a_t^{\text{hand}} = h_{t+1}. \quad (4)$$

Rather than regressing h_{t+1} directly, the frozen encoder first maps the recent hand history to $(\mu_t, \sigma_t) = E_\phi(H_t)$. The behavior cloning policy receives the observation and the prior statistics, and predicts

$$(\hat{a}_t^{\text{arm}}, \Delta \hat{z}_t) = \pi_\psi^{\text{BC}}(o_t, \mu_t, \sigma_t), \quad \hat{h}_{t+1} = D_\theta(\mu_t + \Delta \hat{z}_t). \quad (5)$$

The supervised objective matches the demonstrated arm command and decoded hand target:

$$\mathcal{L}_{\text{BC}} = \|\hat{a}_t^{\text{arm}} - a_t^{\text{arm}}\|_2^2 + \lambda_h \|\hat{h}_{t+1} - h_{t+1}\|_2^2 + \lambda_z \|\Delta \hat{z}_t\|_2^2. \quad (6)$$

The latent offset lets the policy adapt the hand-motion prior to the current visual observation while keeping decoded commands on the learned hand-motion interface. The arm head remains in the native arm coordinate because the arm must express task-level variation such as approach direction, object pose, and placement geometry.

3.3 Stage 3: Residual RL in the Latent Action Space

Stage 3 improves the BC policy through online interaction while preserving the same hand interface. New transitions are appended to an online buffer $\mathcal{D}_{\text{online}}$ and mixed with demonstration data during RLPD updates. Starting from the BC policy and the frozen LMPM decoder, the residual actor

observes $s_t = (o_t, H_t)$ and outputs a residual action $u_t = (\rho_t^{\text{arm}}, \rho_t^z)$ in $\mathcal{A}_{\text{arm}} \times \mathcal{Z}$, rather than in the full raw hand-joint space. During interaction,

$$(a_{t,\text{bc}}^{\text{arm}}, \Delta z_{t,\text{bc}}) = \pi_{\psi}^{\text{BC}}(o_t, \mu_t, \sigma_t), \quad z_{t,\text{bc}} = \mu_t + \Delta z_{t,\text{bc}}, \quad u_t \sim \pi_{\eta}(\cdot | s_t), \quad (7)$$

$$\tilde{a}_t = (a_{t,\text{bc}}^{\text{arm}} + s_{\text{arm}} \rho_t^{\text{arm}}, D_{\theta}(z_{t,\text{bc}} + s_z \rho_t^z)). \quad (8)$$

The executed action is therefore an arm command with a native residual and a decoded hand target with a latent residual. Applying the hand residual in $\mathcal{Z}$ expresses exploration as local corrections to a history-conditioned hand command rather than independent perturbations of every finger joint. We update the residual actor and critic with RLPD-style SAC losses on batches $\mathcal{B} \subset \mathcal{D}_{\text{demo}} \cup \mathcal{D}_{\text{online}}$:

$$\mathcal{L}_Q = \mathbb{E}_{\mathcal{B}}[(Q_{\omega}(s_t, u_t) - y_t)^2], \quad y_t = r_t + \gamma \mathbb{E}_{u' \sim \pi_{\eta}}[\bar{Q}(s_{t+1}, u') - \alpha \log \pi_{\eta}(u' | s_{t+1})], \quad (9)$$

$$\mathcal{L}_{\pi} = \mathbb{E}_{s_t \sim \mathcal{B}, u_t \sim \pi_{\eta}}[\alpha \log \pi_{\eta}(u_t | s_t) - Q_{\omega}(s_t, u_t)]. \quad (10)$$

Sparse task rewards are produced by the visual classifier described in Appendix C.

4 Experiments

To evaluate whether a learned latent motion prior can make real-world dexterous policy improvement more reliable, we ask three questions: (1) whether the learned hand-action interface improves imitation learning over raw, linear, and discrete alternatives; (2) whether the same interface supports stable and efficient online residual RL on real hardware; and (3) which components of LMPM are responsible for the gains.

4.1 Real-World System and Tasks

Our setup uses a Franka Research 3 arm with a Ruiyan dexterous hand and two RGB cameras: a front-view RealSense D435 and a wrist-mounted RealSense D405. Both the visual reward classifier and the visuomotor policy receive the same dual-view RGB observations. Demonstrations are collected with human-in-the-loop teleoperation: a Synglove commands finger motion, and a SpaceMouse commands TCP translation and rotation. Additional implementation, reward, and teleoperation details are provided in Appendices B–D. During online RL and evaluation, the TCP pose is randomized around a nominal start pose to test spatial generalization.

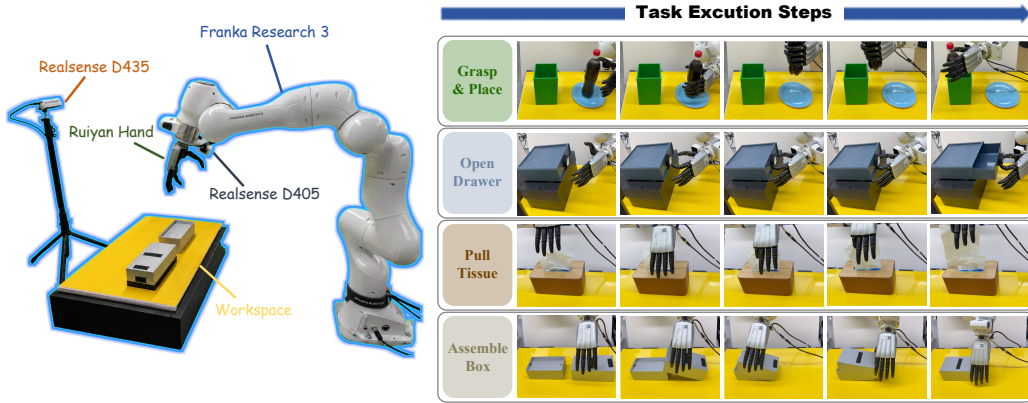


Figure 3: Real-world setup and task scenes. The setup uses a Franka Research 3 arm, a Ruiyan dexterous hand, a front-view RealSense D435 camera, and a wrist-mounted RealSense D405 camera; the right panel shows execution-flow illustrations for the four tasks.

We evaluate on four representative real-world tasks (Figure 3). Each requires sustained hand-object contact during execution, so small finger jitter or abrupt command jumps can break contact, tilt the object, or cause drops. Grasp & Place requires coordinated arm placement while keeping a bottle upright in a narrow, tall box; Open Drawer requires locating and pinching a small drawer handle; Pull Tissue involves a compliant object with changing geometry; and Assemble Box is a long-horizon insertion task where early contact errors can lead to failure or stalled execution.

4.2 Comparison of Action Spaces

We compare four hand-action interfaces under the same imitation-to-residual-RL pipeline. The Raw interface predicts hand commands directly in the original joint space. PCA is a classical linear compression method that reduces the hand-action dimension through a linear transform. VQ-VAE (DQ-RISE) uses the DQ-RISE algorithm [5] to represent hand actions with a discrete codebook. LAMP (Ours) uses the proposed history-conditioned continuous latent action interface. Figure 5 summarizes the corresponding action-interface designs. We evaluate these four interfaces under the imitation-learning framework described in Section 3.2 and perform online post-training with the reinforcement-learning algorithm in Section 3.3. We use the same offline demonstrations and training hyperparameters for all methods; adapter details are provided in Appendix B.

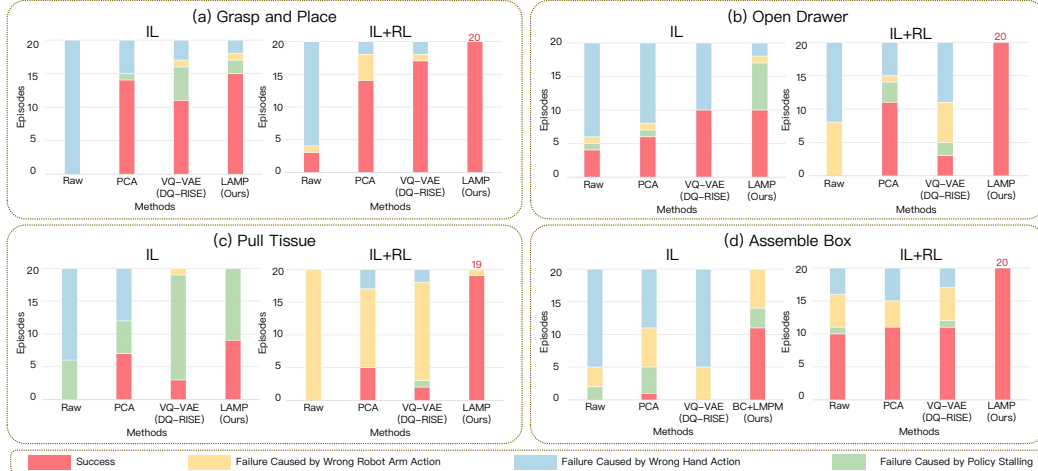


Figure 4: Real-world evaluation on four dexterous tasks. Each bar reports success counts and failures categorized as arm-action error, dexterous-hand error, or stalled execution.

4.2.1 Imitation Learning Performance

From the results in Figure 4, the Raw interface suffers from the high dimensionality of the hand-action regression target: during task execution, the hand motion jitters noticeably, often failing to establish and maintain contact with the object. After PCA linearly compresses the hand action to two dimensions, imitation learning improves substantially over the Raw interface on all four tasks, showing that a compact hand-action representation makes dexterous policy learning easier. VQ-VAE (DQ-RISE) uses a discrete hand-action representation. Although this greatly reduces the action dimension, the hand often switches frequently among nearby actions during execution, causing visible jitter and non-smooth motion. It can perform reasonably well on tasks that require precise finger opposition, but in rigid-object grasping the jitter can easily make the object slip. The remaining gap from PCA to LAMP suggests that compression alone is not sufficient: PCA provides fixed linear hand coordinates, whereas the pretrained history-conditioned encoder in LMPM predicts a latent prior center from recent hand motion, making hand-action continuity directly available to the visuomotor policy. With this interface, LAMP yields stable and smooth hand motion, reliable object contact, and the highest success rate.

4.2.2 Reinforcement Learning Performance

Real-world residual RL is most sample-efficient when exploration remains local to the imitation policy: residuals must improve the behavior without repeatedly breaking contact, so the replay buffer can keep receiving successful or near-successful rollouts. The Raw variant explores in the original hand-action space and starts from a low initial success rate, so RL exploration is extremely inefficient and difficult to converge. PCA reduces the dimension, but its fixed linear projection cannot reparameterize the irregular hand-motion manifold: residuals along retained components

can still decode into contact-breaking finger motion. Although VQ-VAE (DQ-RISE) and LAMP substantially improve imitation learning, their nonlinear interfaces may spread many similar hand motions across a broad latent or code region; without task feedback, the imitation policy can remain in a low-progress part of this region and cause stalled executions. During online RL, task rewards provide a directional signal for moving out of these local regions. VQ-VAE can improve with RL, but codebook switches still introduce gesture jumps that limit final evaluation success. By contrast, LAMP uses a continuous decoder and a history-conditioned prior to organize demonstrated hand motions into a better-conditioned latent space, giving residual RL a smoother, more contact-preserving neighborhood for reward-guided refinement. As a result, RL reaches or approaches 100% success on the tasks. Appendix E provides qualitative examples of successful executions and representative failure cases, and Appendix F quantifies the command-variation differences behind these contact-stability observations.

4.3 Action-Interface Ablations

To isolate which parts of the learned interface matter, we ablate two design choices in the full LMPM model: the compact latent bottleneck and the history-conditioned prior center. Figure 5 summarizes the corresponding architectures together with the baselines, and Table 1 reports the imitation and online RL results, including Raw BC as the raw-space reference.

Both LMPM design choices matter. Expanding the latent dimension usually lowers final performance, suggesting that a larger hand space still burdens arm-hand coordination and on-line exploration. Without the history-conditioned encoder, the visual policy predicts latent points without a local prior center, which weakens imitation initialization; residuals are still applied through the decoder and remain tied to the demonstrated motion manifold, so training is more stable than raw hand control. Raw BC performs poorly across tasks. Additional failure-case analysis for the ablations is provided in Appendix E.

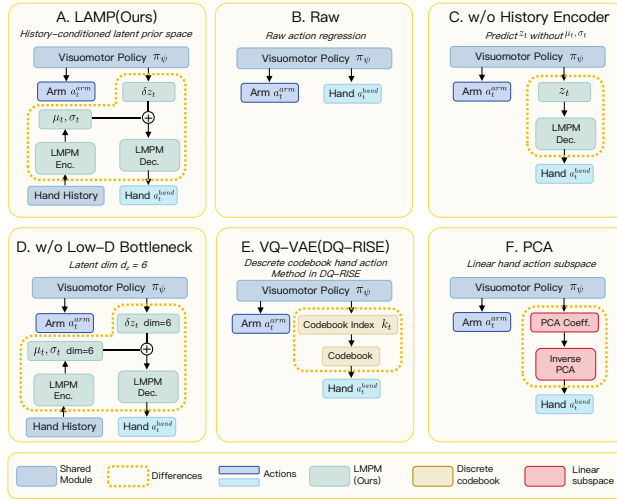


Figure 5: Network architectures for our method, baselines, and ablations.

Variant	Grasp & Place		Open Drawer		Pull Tissue		Assemble Box	
	IL	RL	IL	RL	IL	RL	IL	RL
Full LMPM	75%	100%	50%	100%	45%	95%	55%	100%
w/o low-dimensional bottleneck	40%	35%	65%	85%	15%	80%	5%	20%
w/o history-conditioned encoder	70%	95%	35%	90%	40%	60%	15%	50%
Raw BC (no LMPM)	0%	15%	20%	0%	0%	0%	0%	0%

Table 1: Ablations of LMPM with a raw-space reference. Each task reports imitation-learning (IL) success and final online residual-RL (RL) success.

4.4 Action-Flow Visualization

Figure 6 visualizes the latent action flow during real-world rollouts, showing how actions evolve in the learned hand space across the three stages of LAMP. The history-conditioned prior keeps the hand action on a feasible motion manifold, the IL offset supplies the main vision-guided displacement, and the RL residual makes local online adjustments in the same latent coordinates. When IL leaves a

larger gap, as in Grasp & Place and Pull Tissue, the residual shifts the action toward nearby regions that better complete the task. When IL is already close to success, as in Open Drawer and Assemble Box, the residual remains small and structured. This keeps exploration local while preserving enough freedom to improve contact-rich behaviors.

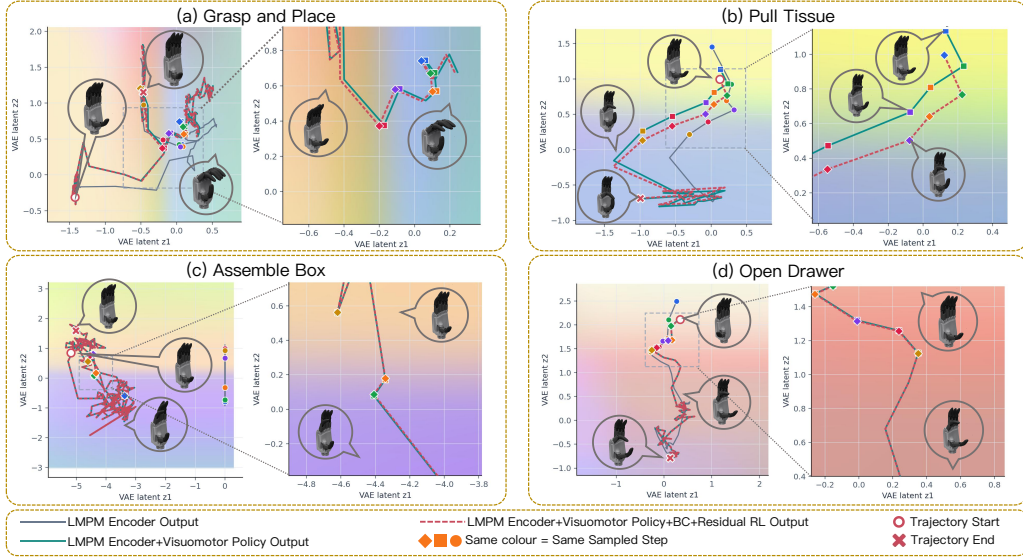


Figure 6: Latent action-flow visualization for the four tasks. Gray trajectories show the LMPM encoder output, teal trajectories show the latent action after adding the IL-predicted latent offset, and red dashed trajectories show the final latent action after applying the RL latent residual. Hand icons show decoded hand gestures at representative latent locations, and markers with the same color indicate the same sampled timestep.

5 Limitations

Our current LMPM is trained from task-specific offline hand-motion data. This makes the latent interface well matched to each task’s contact patterns, but deploying LAMP to a new task still requires suitable hand-motion trajectories and prior pretraining. Learning a multi-task or more general hand-motion prior will likely require broader datasets and higher-capacity models, which we leave for future work.

We validate the latent-prior idea on a 6-DoF hand and map the raw hand space to a 2-D latent space. A bimanual manipulation study with 15-DoF Cyberglove measurements found that the first four PCs explain more than 95% of postural variability [32], suggesting that higher-DoF hands may also admit compact hand-motion coordinates. Validating LMPM on more dexterous hands and broader contact regimes remains future work.

6 Conclusion

We presented LAMP, a real-world dexterous learning framework built around LMPM, a history-conditioned latent motion prior for hand actions. LAMP exposes high-dimensional hand motion through a compact, continuous, and decodable interface shared by the full imitation-to-reinforcement-learning pipeline: behavior cloning predicts offsets around the current prior, and residual reinforcement learning refines the same latent hand action before decoding. This design simplifies high-dimensional hand-action prediction during imitation learning and keeps online exploration close to demonstrated, contact-consistent hand motion. Across four real-world tasks, LAMP outperforms raw, linear, and discrete hand-action representations, supporting latent motion priors as a practical action interface for real-robot dexterous manipulation.

References

- [1] T. Lin, K. Sachdev, L. Fan, J. Malik, and Y. Zhu. Sim-to-real reinforcement learning for vision-based dexterous manipulation on humanoids, 2025. Published at CoRL 2025.
- [2] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In *Robotics: Science and Systems*, 2023.
- [3] C. Chi, S. Feng, Y. Du, Z. Xu, E. Cousineau, B. Burchfiel, and S. Song. Diffusion policy: Visuomotor policy learning via action diffusion. In *Robotics: Science and Systems*, 2023.
- [4] Y. Ze, G. Zhang, K. Zhang, C. Hu, M. Wang, and H. Xu. 3D diffusion policy: Generalizable visuomotor policy learning via simple 3D representations. In *Robotics: Science and Systems*, 2024.
- [5] Y. Feng, H. Fang, Y. He, J. Chen, C. Wang, Z. He, R. Liu, and C. Lu. Learning dexterous manipulation with quantized hand state, 2026. URL <https://arxiv.org/abs/2509.17450>.
- [6] C. von Hofsten. An action perspective on motor development. *Trends in Cognitive Sciences*, 8(6):266–272, 2004. doi:10.1016/j.tics.2004.04.002.
- [7] E. Thelen. Motor development: A new synthesis. *American Psychologist*, 50(2):79–95, 1995. doi:10.1037/0003-066X.50.2.79.
- [8] M. Santello, M. Flanders, and J. F. Soechting. Postural hand synergies for tool use. *Journal of Neuroscience*, 18(23):10105–10115, 1998.
- [9] M. T. Ciocarlie and P. K. Allen. Hand posture subspaces for dexterous robotic grasping. *The International Journal of Robotics Research*, 28(7):851–867, 2009.
- [10] W. Zhou, S. Bajracharya, and D. Held. PLAS: Latent action space for offline reinforcement learning. In *Conference on Robot Learning*, 2020.
- [11] A. Allshire, M. Mittal, V. Lodaya, V. Makoviychuk, D. Makoviychuk, F. Widmaier, M. Wuthrich, S. Bauer, A. Handa, and A. Garg. LASER: Learning a latent action space for efficient reinforcement learning. In *IEEE International Conference on Robotics and Automation*, 2021.
- [12] S. Lee, Y. Wang, H. Etukuru, H. J. Kim, N. M. M. Shafiullah, and L. Pinto. Behavior generation with latent actions, 2024. URL <https://arxiv.org/abs/2403.03181>.
- [13] C. Puang et al. PCHands: PCA-based hand pose synergy representation on manipulators, 2025.
- [14] K. Pertsch, Y. Lee, and J. J. Lim. Accelerating reinforcement learning with learned skill priors, 2020. URL <https://arxiv.org/abs/2010.11944>.
- [15] C. Lynch and P. Sermanet. Learning latent plans from play. In *Conference on Robot Learning*, 2019.
- [16] J. Liang et al. CLAM: Continuous latent action models for robot learning from unlabeled demonstrations, 2025.
- [17] B. Bauer et al. Latent action diffusion for cross-embodiment manipulation, 2025.
- [18] P. Wu et al. Discrete policy: Learning disentangled action space for multi-task robotic manipulation, 2024.
- [19] K. Pertsch et al. FAST: Efficient action tokenization for vision-language-action models, 2025.
- [20] J. Yang et al. VQ-ACE: Efficient policy search for dexterous robotic manipulation via action chunking embedding, 2024.

- [21] G. Jiang, Y. Liang, J. Ye, J.-Y. Huang, C. Jing, R. Duan, P. Abbeel, X. Wang, and X. Zou. Cross-hand latent representation for vision-language-action models, 2026. URL <https://arxiv.org/abs/2603.10158>.
- [22] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, X. Chen, K. Choromanski, T. Ding, D. Driess, A. Dubey, C. Finn, et al. RT-1: Robotics transformer for real-world control at scale. In *Robotics: Science and Systems*, 2023.
- [23] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, et al. RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, 2023.
- [24] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, T. Lampe, P. Sanketi, et al. OpenVLA: An open-source vision-language-action model. In *Conference on Robot Learning*, 2024.
- [25] Octo Model Team. Octo: An open-source generalist robot policy. In *Robotics: Science and Systems*, 2024.
- [26] P. J. Ball, L. Smith, I. Kostrikov, and S. Levine. Efficient online reinforcement learning with offline data. In *International Conference on Machine Learning*, 2023.
- [27] J. Luo et al. SERL: A software suite for sample-efficient robotic reinforcement learning. In *IEEE International Conference on Robotics and Automation*, 2024.
- [28] J. Luo et al. Precise and dexterous robotic manipulation via human-in-the-loop reinforcement learning. In *Conference on Robot Learning*, 2024.
- [29] Y. Zang et al. RLinf-USER: A unified and extensible system for real-world online policy learning in embodied AI, 2026.
- [30] Y. Zang et al. RLinf-VLA: A unified and efficient framework for reinforcement learning of vision-language-action models, 2025.
- [31] D. Yu, K. Lei, Z. Jiang, J. Pan, and H. Xu. Beyond action residuals: Real-world robot policy steering via bottleneck latent reinforcement learning, 2026.
- [32] N. Jarrassé, A. T. Ribeiro, A. Sahbani, W. Bachta, and A. Roby-Brami. Analysis of hand synergies in healthy subjects during bimanual manipulation of various objects. *Journal of NeuroEngineering and Rehabilitation*, 11(1):113, 2014. doi:10.1186/1743-0003-11-113.

A Off-Manifold Exploration Analysis

Online residual RL should explore near the demonstrated hand-motion manifold: moving too far away from it often breaks contact before the reward signal can guide recovery. We therefore measure how much of a small exploration step leaves this manifold. For each method, we match the decoded hand-action displacement budget, denoted Disp , and compute the increase in nearest-neighbor distance to the demonstrated hand actions, denoted ΔNN . The ratio $\Delta\text{NN}/\text{Disp}$ measures the fraction of the step that points away from the data-supported motion manifold.

This normalization separates the size of an exploration step from its direction. A value near 0 means that the decoded perturbation mostly moves along nearby demonstrated motions; a value near 1 means that the same displacement is almost entirely off-manifold. Figure A.1 reports the ratio at $\text{Disp} = 0.2$ for the continuous action interfaces. Raw exploration is consistently far from the manifold, and PCA remains substantially off-manifold because its fixed linear coordinates do not follow the curved hand-motion structure. LAMP keeps the ratio lowest on most tasks, showing that its continuous latent residuals provide a better local neighborhood for real-robot exploration.

Fraction of exploration that leaves the manifold (matched $\text{Disp} = 0.20$)

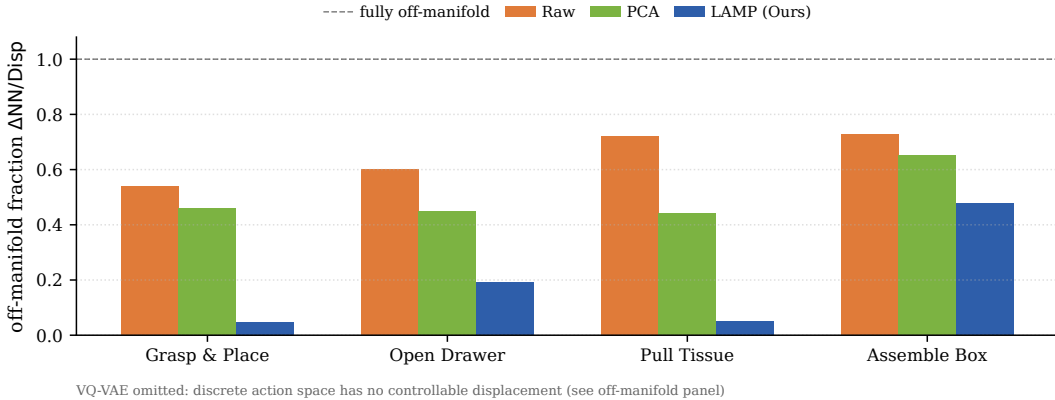


Figure A.1: **Fraction of exploration that leaves the manifold** at a matched displacement budget ($\text{Disp} = 0.2$). Bars show $\Delta\text{NN}/\text{Disp}$; 1.0 (dashed) is fully off-manifold. LAMP keeps the smallest off-manifold fraction on most tasks. VQ-VAE is omitted here because its discrete action space cannot realise a controllable displacement budget; its behaviour is shown in the off-manifold panel instead.

B Implementation Details

Unless otherwise specified, all action-interface variants use the same demonstrations, camera inputs, arm-action parameterization, behavior-cloning backbone, residual-RL algorithm, and evaluation protocol.

B.1 Observations and Actions

At each control step, the policy receives the front and wrist RGB images, the current 12-D robot state, and the most recent eight hand targets. The robot action is represented as a 6-D arm command concatenated with a 6-D Ruiyan-hand command. Demonstrations store the hand command as an absolute target. During robot execution, the environment wrapper converts the decoded absolute hand target into the low-level hand delta used by the hardware interface.

Behavior cloning is trained as one-step target prediction. Given the observation at time t and the current action/state a_t , the supervised target is the next 12-D action a_{t+1} . At the start of a trajectory, missing hand-history entries are padded with the first recorded hand target.

B.2 LMPM Pretraining and Behavior Cloning

LMPM is trained only on the hand-motion component of the demonstrations. Its encoder maps an 8-step hand-target history to a Gaussian latent prior in $\mathbb{R}^2$, and its decoder maps a latent sample to the next 6-D absolute hand target. Both encoder and decoder are MLPs with hidden width 256. We train with a KL weight of 10^{-3} .

All action interfaces are built from the same small offline set for each task: 50 demonstrations for Grasp & Place, 20 for Open Drawer, 20 for Pull Tissue, and 30 for Assemble Box. We use a 9:1 split of these demonstrations throughout supervised training: the training split is used to fit LMPM from hand-motion sequences and to train the BC policy, while the held-out split is used for LMPM testing and BC validation. All action-interface variants use the same split.

The BC policy uses one frozen ImageNet-pretrained ResNet-18 encoder for each RGB view. Image features are concatenated with the robot state and hand-history features, and the CoreActionHead MLP has hidden sizes [512, 512, 256]. For LAMP, the head predicts a 6-D arm command and a 2-D latent offset. The hand target is decoded as $D_\theta(\mu_t + \Delta z_t)$, where μ_t is the frozen LMPM prior mean computed from the hand history. The supervised hand loss is applied after decoding in the 6-D hand-target space.

Setting	LMPM pretraining	BC policy
Input history	8 hand targets	8 hand targets
Output target	next 6-D hand target	next 12-D robot action
Network	MLP encoder/decoder, width 256, $d_z = 2$	frozen ResNet-18 per view; MLP head [512, 512, 256]
Optimizer	AdamW	AdamW
Batch size	256	128
Learning rate	2×10^{-3}	5×10^{-4}
Training steps	20k	20k
Warmup	500 LR steps; 2000 KL steps	500 LR steps
Noise augmentation	hand-history std 0.01	arm-state and hand-history std 0.10
Gradient clip	1.0	1.0

Table B.1: Training settings for LMPM and the behavior-cloning policy.

B.3 Residual RLPD and Evaluation

For online improvement, we freeze the BC policy and train a residual SAC/RLPD agent. The residual actor is a tanh-squashed Gaussian policy in the BC core-action space. For LAMP, this space is 8-D: six arm coordinates and two latent hand coordinates. The residual is added to the stopped-gradient BC core action, then the hand component is decoded by the corresponding hand interface in Appendix B.4. The critic is trained on the executed 12-D environment action.

For residual RL, the full offline demonstration set described in Appendix B.2 is loaded into the demonstration buffer before online interaction begins, while newly collected robot transitions are appended to the online replay buffer. The learner samples half of each batch from online replay and half from the demonstration buffer. With critic-to-actor ratio 2, each learner iteration performs one critic-only update followed by one update of the critic, actor, and entropy temperature. Real-world evaluation uses randomized task starts with fixed policy parameters. Sparse rewards and evaluation success labels are produced by the visual classifier in Appendix C.

Within each task, all action-interface variants use the same online training budget: 20k steps for Grasp & Place, 30k for Open Drawer, 40k for Pull Tissue, and 25k for Assemble Box.

Setting	Value
Actor / critic MLP	[256, 256], tanh activations, layer normalization
Critic ensemble size	2
Discount	0.97
Batch size	256 = 128 online + 128 demonstration
Critic-to-actor ratio	2
Replay capacity	200k transitions
Training starts	100 online transitions
Network publish interval	50 learner steps
Initial temperature	10^{-2}

Table B.2: Residual-RLPD hyperparameters shared by all action-interface variants.

B.4 Compared Action Interfaces

Only the hand-action interface changes across Raw, PCA, VQ-VAE, and LAMP. The arm command is always predicted in the native 6-D arm-command space. The resulting hand representation is decoded or transformed back to a 6-D absolute hand target before being concatenated with the arm command.

Interface	Policy hand output	Conversion to 6-D hand target
Raw	6-D hand target	identity
PCA	2-D PCA coordinate	inverse PCA transform fitted on demonstration hand targets
VQ-VAE	16-way residual-VQ code	two residual quantizers with four codes each
LAMP	2-D latent offset Δz_t	frozen LMPM decoder $D_\theta(\mu_t + \Delta z_t)$

Table B.3: Hand-action interfaces used with the same BC and residual-RL pipeline.

C Visual Reward Classifier

We train a task-specific binary visual classifier for sparse online rewards and evaluation success labels. The classifier takes the processed front and wrist RGB views used by the policy, denoted by the `global` and `wrist` image keys, and does not use proprioceptive state. The two view features are concatenated and mapped to a single success logit $f_{\text{cls}}(o)$; Table C.1 gives the architecture and training settings.

Classifier data are collected separately for each task in the real environment. The operator marks successful states during collection, and unmarked visited states from the same sessions are saved as negative examples. Training samples half of each minibatch from the positive replay buffer and half from the negative replay buffer. Let $p_{\text{cls}}(o) = \sigma(f_{\text{cls}}(o))$. We optimize

$$\mathcal{L}_{\text{cls}} = -\mathbb{E}_{(o,y)} [y \log p_{\text{cls}}(o) + (1 - y) \log(1 - p_{\text{cls}}(o))]. \quad (\text{C.1})$$

Training uses the task-specific camera crops defined in the environment config, without stochastic image crop augmentation.

During online interaction, the environment wrapper evaluates the classifier after each robot step and returns

$$r_t = \mathbb{I}[p_{\text{cls}}(o_{t+1}) > 0.90]. \quad (\text{C.2})$$

When $r_t = 1$, the wrapper also terminates the episode and sets the success flag in the environment info. This 0.90 threshold is used for online RL rewards and evaluation success labels.

Item	Setting
Input	global and wrist RGB views; no proprioception
Visual encoder	One frozen ImageNet-pretrained SERL ResNet-10 per view
Pooling	Spatial learned embeddings with 8 blocks and a 256-D bottleneck
Classifier head	Dense 256, dropout 0.1, layer norm, ReLU, Dense 1
Training labels	Operator-marked success states as positives; unmarked visited states as negatives
Batching	256 total: 128 positive and 128 negative examples
Optimizer	Adam with learning rate 10^{-4}
Training length	250 epochs
Deployment rule	Sparse reward and success when $p_{\text{cls}}(o) > 0.90$

Table C.1: Visual reward classifier settings used for online RL and evaluation.

D Teleoperation and Data Collection

Demonstrations are collected with the teleoperation devices in Figure D.1. The SpaceMouse provides the 6-D end-effector command for TCP translation and rotation, while the glove provides the 6-D Ruiyan-hand command. In the real-robot environment, the teleoperation wrapper reads both devices at each control step. Nonzero SpaceMouse motion overrides the arm command. Holding the SpaceMouse right button enables hand teleoperation: the wrapper subtracts the current glove baseline from the measured glove angles, applies the configured hand-joint limits when present, and concatenates the resulting hand delta with the SpaceMouse arm command. When the right button is not held, only the arm part is replaced and the current hand command is left unchanged.

During demonstration recording, the script sends a zero nominal action to the environment and stores the action actually executed by the wrapper from `info["intervene_action"]`. Each saved transition contains synchronized RGB observations, proprioceptive state, the executed 12-D arm-hand action, reward and terminal flags, and environment info.



Figure D.1: Teleoperation devices. The glove controls the Ruiyan hand; the SpaceMouse controls the end-effector pose.

E Qualitative Success and Failure Cases

Figure E.1 shows representative evaluation frames for the four real-world tasks. For each task, we compare a successful execution with three recurring failure modes: arm-action errors, dexterous-hand errors, and stalled executions. Each row uses a fixed crop window, so the comparison focuses on the robot-object state rather than camera framing.

We assign a failure to the component that prevents continued task progress in the rollout. Arm failures occur when the end-effector approaches with an unsuitable pose or path, such as missing the drawer pull direction, approaching the tissue off center, or reaching the bin or lid from a pose that cannot complete the interaction. Hand failures occur after the arm reaches the relevant region but the fingers do not maintain the needed contact: the object slips, the drawer edge is not secured, the tissue is pressed instead of pinched, or the lid is tipped away. Stall failures are rollouts that remain near the object but stop making progress before satisfying the task condition.

These examples complement the quantitative failure counts in the main results. Successful executions require both a useful arm trajectory and stable finger contact. Once contact is lost or the object is perturbed into an unfavorable pose, recovery is difficult within the same rollout, which is why contact-preserving hand exploration is important for online refinement.

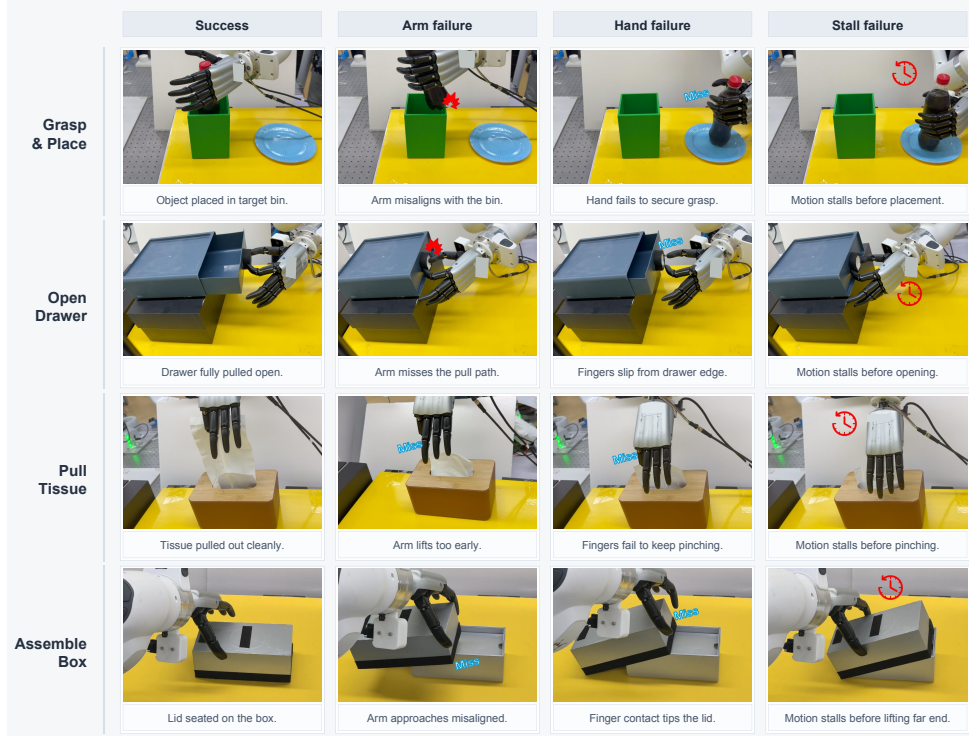


Figure E.1: Qualitative success and failure examples from real-world evaluation rollouts. Rows correspond to tasks; columns show a successful rollout and representative arm, hand, and stall failures.

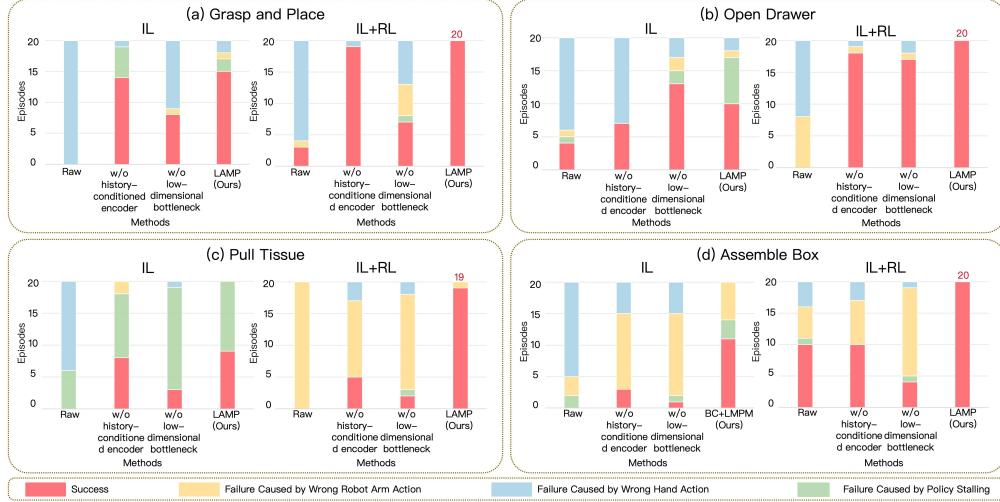


Figure E.2: Ablation outcome breakdown on four real-world tasks. In each task panel, the two subplots correspond to imitation learning (IL) and the final policy after residual RL (IL+RL); the x-axis lists methods and the y-axis reports evaluation episodes. Each stacked bar partitions the 20 evaluation episodes into successes and failures caused by wrong arm action, wrong hand action, or policy stalling. Raw directly regresses the original hand action, w/o history-conditioned encoder removes the prior center computed from recent hand motion, and w/o low-dimensional bottleneck removes the 2-D latent bottleneck while keeping the remaining pipeline unchanged.

Figure E.2 further breaks down the main ablation results by failure category. The low-dimensional bottleneck helps arm-hand coordination rather than only improving the fingers in isolation. After residual RL, removing the bottleneck leaves many arm-action failures, most visibly on Pull Tissue and Assemble Box, whereas the full LMPM model has only one arm-action failure across the four tasks. Since the visuomotor policy predicts arm and hand commands jointly, a less compact hand representation also changes the learning problem faced by the arm branch: the policy must fit and explore a larger coupled action space, and the end-effector is more often driven into poses from which the remaining hand motion cannot recover the task.

The history-conditioned encoder has a less category-specific but still visible role. Without the encoder, the policy no longer receives a local prior center from recent hand motion and must predict latent hand commands without this phase reference. The decoder still keeps actions on a learned hand-motion manifold, so residual RL can recover well on the shorter Grasp and Place and Open Drawer tasks. On Pull Tissue and Assemble Box, however, the same ablation leaves mixed arm and hand failures after RL, consistent with poorer timing between the arm approach and the hand contact state. In contrast, the full LMPM interface provides both a compact bottleneck and a history-conditioned local coordinate system, making post-training failures sparse rather than shifting them between components.

F Action Smoothness Analysis

In contact-rich dexterous manipulation, hand-command jitter can break established contacts, induce slips or drops, and make real-robot exploration less recoverable. We analyze the hand-action histories saved during IL and RL evaluation rollouts. To separate high-frequency jitter from deliberate hand opening or closing, we use the second-order hand-target variation

$$J = \frac{1}{T-2} \sum_{t=1}^{T-2} \|a_{t+2}^{\text{hand}} - 2a_{t+1}^{\text{hand}} + a_t^{\text{hand}}\|_2, \quad (\text{F.1})$$

where a_t^{hand} is the executed 6-D absolute hand target at time step t . Lower values indicate smoother hand motion, while large values correspond to rapid changes in the direction or magnitude of the hand command.

Figure F.1 visualizes the same metric in two ways. The top row shows final-RL evaluation rollouts as a jitter barcode: each row is one rollout and red bands indicate high second-order command variation. Raw and VQ-VAE policies exhibit frequent high-jitter bands, while LAMP keeps most rollouts close to the low-jitter range. The bottom plot aggregates IL and RL rollouts across tasks after normalizing each task by its Raw-IL jitter score.

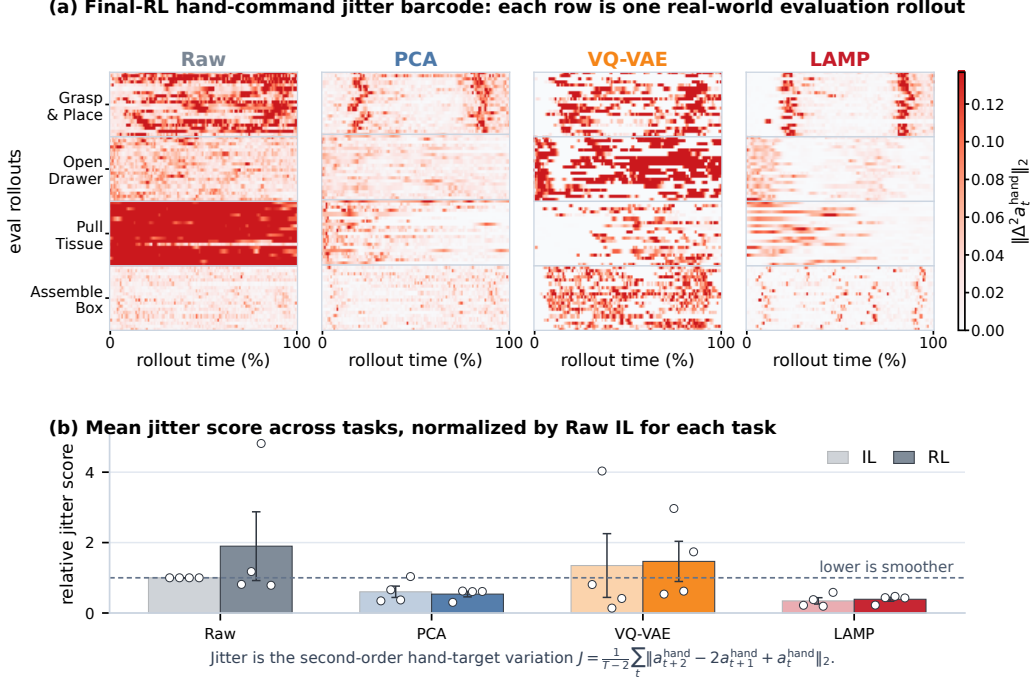


Figure F.1: Hand-command jitter during real-world evaluation. Top: final-RL jitter barcode. Within each method panel, the vertical axis stacks individual evaluation rollouts grouped by task; each horizontal row is one rollout, not a hand dimension. The horizontal axis is normalized rollout time, and color denotes the per-time-step term $\|\Delta^2 a_t^{\text{hand}}\|_2 = \|a_{t+2}^{\text{hand}} - 2a_{t+1}^{\text{hand}} + a_t^{\text{hand}}\|_2$ used in Eq. (F.1). The rollout-level jitter score J is the average of these colored values over time. Bottom: task-normalized jitter scores for IL and RL evaluation rollouts; each task is normalized by its Raw-IL score, so lower values indicate smoother hand commands.